

SEO Package

SEO Title

n8n IF Node Explained: Fix the String vs Number Comparison Bug That Breaks Branches

SEO Subtitle

A practical walkthrough of n8n's IF node, from condition anatomy to the silent type-mismatch bug that makes true/false branching fail without an error.

Featured Quote

"n8n only executes the branch that fired; the other one just never runs, so there's nothing to show. No error, no red X, just silence."

Meta Description

Learn how n8n's IF node routes workflows on real conditions, why "50" and 50 break branches silently, and how to build multi-condition logic that actually works.

URL Slug

n8n-if-node-string-vs-number-comparison-bug

Keywords

n8n IF node, n8n tutorial, workflow automation, no-code automation, n8n conditions, n8n branching, n8n Switch node, string vs number comparison, n8n type coercion, n8n Slack integration, automation logic, conditional workflows, n8n AND OR conditions, n8n debugging, low-code automation, n8n beginners guide, workflow orchestration, n8n data types, business process automation, n8n error handling

Tags

n8n, automation, no-code, workflow-automation, IF-node, low-code, integration, Slack-automation, business-automation, tutorial, beginners, data-types, debugging, workflow-design, tech-education

Why "50" and 50 Break Your IF Node

Introduction

Most people's first n8n workflow looks the same: a trigger fires, and one action runs every single time. It works fine in a demo. It falls apart the moment real data hits it, because real data isn't uniform. A \$50,000 deal and a \$5 deal are not the same event, and a workflow that treats them identically will eventually get ignored by the humans downstream of it.

This is where conditional branching becomes non-negotiable, and it's also where a surprising number of automations quietly break. The IF node is the simplest branching tool n8n offers, and it looks foolproof: check a field, get true or false, move on. But there's a specific, common bug hiding inside that simplicity — a string being compared to a number — that produces no error, no warning, and no obvious symptom. The workflow just silently takes the wrong path.

If you're building automations that gate expensive actions (a Slack ping, an email, a paid API call) behind a condition, understanding exactly how the IF node evaluates data isn't optional. This is also a pattern that shows up constantly in technical interviews and system design discussions: a single trigger needing to behave differently based on payload contents is the same problem whether you're wiring no-code nodes or writing a Python `if/elif` chain against a webhook payload.

Problem Overview

You have a single trigger — say, a new deal record — and you need the automation's behavior to depend on the data in that record, not just on the fact that the trigger fired. Specifically:

- Deals above a certain dollar amount should notify a human immediately.
- Deals below that amount should be logged quietly, with no interruption.
- The rule might need more than one condition (amount AND region, for example).
- The field you're checking might arrive as the wrong data type, which can make an otherwise-correct condition evaluate incorrectly without throwing any error.

The goal is a workflow that reliably splits execution into different paths based on the actual content of the data, and that fails loudly (or not at all) rather than failing silently.

Example

Say every new deal comes in as a record with two fields: `deal_amount` and `region`.

Case A: `deal_amount = 5000`, `region = "US"` A condition like "deal_amount greater than 1000" evaluates to `true`. The workflow follows the true branch, which is wired to a Slack node — your sales manager gets pinged.

Case B: `deal_amount = 50`, `region = "US"` The same condition evaluates to `false`. The workflow follows the false branch, wired to a logging step. No Slack message. No error. The Slack node in the canvas simply never lights up or gets a checkmark, because it never ran.

Case C: `deal_amount = "1000"` (a string, not a number), condition is "greater than 1000" This is the trap. You'd expect `true`, since 1000 is not greater than 1000... actually you'd expect this to correctly evaluate as `false` either way — but the real danger case is `deal_amount = "5000"` compared against `1000`. Depending on how the comparison is typed, a string "5000" can silently fail a numeric "greater than" check that a real number 5000 would pass. The condition doesn't throw an error; it just returns the wrong boolean.

Intuition

Before touching the node, think about what a condition actually is. It isn't a general-purpose script — it's closer to a single yes/no gate. A useful mental model: an IF node is a bouncer at a club door. One look, one decision, then you're in or you're not. There's no in-between and no partial credit.

That mental model does two things for you. First, it tells you where the "expensive" action belongs — directly on the branch that fires only when the condition truly holds. You don't sprinkle side effects across both branches; you put the costly, visible action (a notification, a paid API call) exactly on the branch that's supposed to be rare or significant, and you let the other branch do something cheap and silent, like logging.

Second, it forces you to ask a question before you ever click "execute": what type is this data, really? A bouncer checking a birthdate can't do their job if the ID is handed over as a smudged photocopy — they need the value in a comparable form. n8n's IF node has the same requirement. If a field is technically a string that *looks* like a number, a numeric comparison operator can behave unpredictably, and there's no red flag telling you it happened. This is the single most common way IF-node logic silently drifts

from what you intended, and it's why experienced automation builders get in the habit of checking (or explicitly forcing) a field's type before wiring a condition to it, rather than assuming the trigger delivers clean data.

Once you internalize "one field, one comparison, one boolean, no in-between," building correct conditions becomes mechanical. The remaining skill is just being suspicious of data types and knowing when two conditions need to be combined with AND versus OR.

Brute Force Solution

The brute-force version of "different behavior for different data" is building two (or more) entirely separate workflows, each triggered independently, with the routing logic reimplemented outside n8n — for example, a webhook receiver written in Python that inspects the payload and calls a different n8n webhook URL depending on the value.

Idea: Push the branching decision upstream, before the workflow tool ever sees the data.

Algorithm: 1. Receive the raw payload in custom code (a script, a serverless function). 2. Inspect the relevant field. 3. Forward the payload to Workflow A's webhook or Workflow B's webhook based on that inspection.

Advantages: - Full control over comparison logic, including explicit type coercion. - Easy to unit test outside of n8n.

Disadvantages: - Duplicates infrastructure you already have inside n8n. - Now you're maintaining a second codebase just to make a decision the IF node was built for. - Adds a network hop and a new failure point. - Loses the visual, at-a-glance auditability that's the entire reason to use a workflow tool in the first place.

Complexity: Time is $O(1)$ per request either way — this isn't a performance problem, it's an architecture problem. The "cost" here is maintenance and operational surface area, not runtime.

Optimal Solution

The optimal solution is to let the IF node do exactly the job it's designed for, and be deliberate about the three things every condition is built from:

Part	Question it answers	Example
Field	Which value are you checking?	deal_amount
Operator	How are you comparing it?	greater than, equals, contains
Value	What are you comparing it to?	1000

Step 1 — Wire the true branch to the expensive action. The true output only fires the instant the condition holds. That's exactly where a Slack ping, an email, or any action that interrupts a human belongs — and nothing else needs to sit on that branch.

Step 2 — Wire the false branch to something quiet. Everything that doesn't clear the bar still gets recorded, it just doesn't interrupt anyone. A logging step is enough.

Step 3 — Stack conditions when a single field isn't enough. Real rules rarely hinge on one field. Add a second condition — `deal_amount over 1000` AND `region equals "US"` — and both have to hold. Switch the combinator to OR if only one of the two needs to match.

Step 4 — Force the type before comparing. This is the step that prevents the silent-failure bug. If a field might arrive as a string that looks numeric (a very common outcome from web forms), explicitly coerce it to a number inside the condition rather than trusting the incoming type. n8n exposes this as an option in the condition settings specifically because this failure mode is common enough to need a built-in fix.

Step 5 — Know when IF isn't enough. IF only ever produces two outputs: true or false. The moment a single field needs to drive three or four different outcomes, that's a signal to reach for the Switch node instead of chaining multiple IF nodes together.

Python Code

Here's the equivalent logic expressed in Python, which is useful both for understanding what the IF node does under the hood and for anyone routing this decision through custom code (a webhook handler, a serverless function, or a test harness):

```
from dataclasses import dataclass

@dataclass
class Deal:
    deal_amount: float
    region: str

def should_notify_manager(deal: Deal, threshold: float = 1000, target_region: str = "US") -> bool:
    """Return True if the deal should trigger an immediate notification."""
    # Force numeric comparison explicitly -- mirrors n8n's "force type" option.
    amount = float(deal.deal_amount)
    return amount > threshold and deal.region == target_region

def route_deal(deal: Deal) -> str:
```

```

"""Return which branch this deal takes: 'notify' or 'log'."""
return "notify" if should_notify_manager(deal) else "log"

def handle_deal(raw_payload: dict) -> str:
    # Payloads from web forms often arrive with numeric fields as strings.
    deal = Deal(deal_amount=raw_payload["deal_amount"], region=raw_payload["region"])
    branch = route_deal(deal)

    if branch == "notify":
        send_slack_alert(deal)
    else:
        log_deal(deal)

    return branch

def send_slack_alert(deal: Deal) -> None:
    print(f"[SLACK] New deal ${deal.deal_amount} in {deal.region} needs review")

def log_deal(deal: Deal) -> None:
    print(f"[LOG] Deal recorded: ${deal.deal_amount} in {deal.region}")

```

Code Walkthrough

- `Deal` is a small dataclass standing in for the record your trigger receives — this mirrors the "field" concept from the IF node.
- `should_notify_manager` is the condition itself. `float(deal.deal_amount)` is the critical line: it forces the type conversion explicitly, the same job n8n's "force type" toggle does inside a condition. Without it, a string `"5000"` compared against a numeric `1000` can behave inconsistently depending on how the comparison is evaluated.
- The `and` in the return statement is the AND combinator — both conditions must hold, exactly like stacking two conditions in the IF node.
- `route_deal` is the branch decision itself — the true/false split, made explicit as a string instead of an implicit graph edge.
- `handle_deal` is the trigger handler: it receives the raw payload (where the type problem originates), builds a typed `Deal`, and only then evaluates the condition. This ordering — normalize first, decide second — is the whole fix for the silent bug.
- `send_slack_alert` and `log_deal` are the two branch outcomes, deliberately asymmetric in cost: one interrupts a human, one doesn't.

Dry Run

Input: `raw_payload = {"deal_amount": "5000", "region": "US"}`

1. `handle_deal` builds `Deal(deal_amount="5000", region="US")` — note `deal_amount` is still a string here.
2. `route_deal` calls `should_notify_manager(deal)`.
3. Inside that function, `float(deal.deal_amount)` converts `"5000"` to `5000.0`.
4. `5000.0 > 1000` → `True`. `deal.region == "US"` → `True`. `True and True` → `True`.
5. `should_notify_manager` returns `True`, so `route_deal` returns `"notify"`.
6. `handle_deal` calls `send_slack_alert(deal)`, printing the alert.
7. Function returns `"notify"`.

Now compare: if the `float()` conversion were missing and the comparison were done as a raw string vs. int comparison in a language/tool that doesn't coerce, `"5000" > 1000` could raise an error or silently evaluate in a way that doesn't match numeric intuition — which is precisely the bug the video walks through.

Complexity Analysis

- **Time:** $O(1)$ per deal evaluated — a fixed number of field lookups, one type conversion, and two comparisons regardless of input size.
- **Space:** $O(1)$ — no data structures scale with input; each deal is evaluated independently.
- These bounds hold because the condition never iterates over a collection; it only ever inspects a fixed, small set of fields on a single record.

Alternative Solutions

If you need more than two outcomes from one field (small / medium / large deal, for instance), chaining multiple IF nodes works but gets messy fast — each additional outcome adds another node and another wire to track. The Switch node is the better fit once you're past two branches: it takes one field and routes to N outputs based on multiple matching rules, which is a closer match to a Python `match` statement or `if/elif/elif/else` chain than two IF nodes bolted together.

Edge Cases

- **Missing field:** if `deal_amount` isn't present on the payload at all, `float(deal.deal_amount)` raises a `KeyError` / `TypeError` rather than silently defaulting — decide explicitly whether a missing field should fail loudly or fall back to a default.
- **Non-numeric string:** a value like `"N/A"` passed to `float()` raises `ValueError` — worth validating upstream if the field isn't guaranteed clean.
- **Boundary value:** a deal at exactly the threshold (`deal_amount = 1000`) needs a clear decision on whether "greater than" should be "greater than or equal to."
- **Negative or zero amounts:** make sure the condition doesn't assume all amounts are positive if refunds or corrections can produce negative values.
- **Case sensitivity in the region field:** `"us"` vs `"US"` will fail an exact-match condition silently — normalize casing before comparing.

Common Mistakes

1. Comparing a numeric-looking string against a number without forcing the type, and trusting the result.
2. Wiring extra actions onto the true branch "just in case," turning a clean condition into a tangle of side effects.
3. Assuming a workflow "failed" because a branch's nodes show no checkmark — they simply never executed, which is expected behavior, not an error.
4. Using AND when the intent was OR (or vice versa) when stacking multiple conditions.
5. Chaining several IF nodes for what's really a multi-outcome decision, instead of switching to the Switch node.
6. Not testing both branches with real boundary values before trusting the workflow with live data.

Interview Questions

- How would you design a system where a single event needs different downstream handling based on payload content?
- What's the risk of comparing values of different types, and how do you defend against it in a language like Python versus a no-code tool?
- When would you choose a chain of if/elif statements versus a dictionary-based dispatch or a match statement?
- How do you test that both branches of a conditional pipeline behave correctly, including edge cases at the boundary?
- What's the tradeoff between building branching logic inside a workflow tool versus handling it in custom code upstream?

Similar Problems

- **LeetCode 88 (Merge Sorted Array)** — not branching-related directly, but a good companion for practicing careful boundary-condition handling, the same discipline that catches the string/number bug here.
- **LeetCode 20 (Valid Parentheses)** — another problem where a single comparison type-mismatch (character vs. expected symbol) causes silent logical errors if not checked carefully.
- **Any "route by payload type" system design question** — the underlying pattern (one trigger, multiple behaviors based on data) is identical to designing a webhook dispatcher or event router.

Key Takeaways

- A condition is one field, one operator, one comparison value — nothing more.
- Put expensive, human-interrupting actions strictly on the branch that's supposed to be rare.
- A branch with no checkmark isn't a failure — it simply didn't run.
- Type mismatches (string vs. number) are the most common source of silently wrong branching, and they never throw an error on their own.
- Once one field needs more than two outcomes, reach for a Switch node instead of chaining IFs.

Watch the Video

This gets a lot clearer watching it run live and seeing exactly which nodes light up on each branch. Watch the full walkthrough here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series where real automations and real code get built end to end instead of narrated over slides — n8n workflows, algorithms, and the everyday engineering problems that trip people up in production.

Call To Action

If this saved you from a silent branching bug, drop a like on the video and subscribe on YouTube so future workflow breakdowns land in your feed. For deeper dives and written companions to every episode, subscribe to the Substack. Got a workflow that's misbehaving in a way you can't explain? Comment with what you're seeing — these episodes are picked from what people are actually stuck on.

Social Media

LinkedIn Post

Most automation bugs aren't logic errors — they're type errors wearing a logic-error costume.

Built an n8n workflow this week that routes deals based on amount: big deals ping the sales manager on Slack, small deals log quietly. Straightforward IF node, one condition, two branches.

Then I hit the bug that gets everyone eventually: the form was sending `deal_amount` as a string, `"1000"`, not a number. The condition compared it against a numeric threshold, expected `true`, and silently returned `false` instead. No error. No red X. The workflow just took the wrong branch and nobody noticed until deals stopped generating alerts.

This is a pattern that shows up everywhere, not just in no-code tools — Python, JavaScript, SQL, all of them have some version of "this comparison looked right but the types didn't match." The fix is always the same: normalize types explicitly before comparing, don't assume the data arrived clean.

The other lesson worth internalizing: a branch that doesn't fire isn't a failure state. If your workflow tool shows one path lit up and the other dim with no error, that's the design working correctly — only the branch that matched actually runs.

Full breakdown (with the fix) in the video and companion article. Curious what type-mismatch bugs have burned other people building automations — drop them below.

Twitter/X Thread

1/ Built an n8n workflow this week: big deals ping Slack, small deals get logged quietly. One IF node, one condition. Then it broke in a way that looked like nothing happened.

2/ The setup: `deal_amount > 1000` → notify. Everything else → log. Simple. Tested it with a \$5,000 deal, worked perfectly, lit up the Slack branch.

3/ Then a \$50 deal came through. Correctly took the false branch, logged quietly. No Slack ping. Looked exactly like it should.

4/ Except the Slack node showed no checkmark at all — just grey. First time you see that, it looks like the run failed. It didn't. n8n only executes the branch that actually fired.

- 5/ The real bug showed up later: the form was sending `deal_amount` as a string — `"1000"`, not `1000`. Compared against a numeric threshold, it silently evaluated wrong. No error thrown anywhere.
- 6/ This is the trap with type coercion in any comparison logic — no-code tool or full codebase. `"1000"` and `1000` look identical to a human reading a screen. They are not identical to a comparison operator.
- 7/ The fix: force the type explicitly before comparing. n8n has a setting for exactly this in the condition. In Python, that's just `float(value)` before your comparison — same idea, different syntax.
- 8/ Bigger lesson underneath: IF only gives you two outputs, true or false. The moment you need three or four outcomes off one field, that's your signal to reach for a Switch node instead of chaining IFs.
- 9/ None of this is really about n8n. It's about the general shape of "one trigger, multiple behaviors based on data" — which is the same problem whether you're wiring nodes or writing an `if/elif` chain against a webhook payload.
- 10/ Full write-up + Python equivalent + video walkthrough linked below. If you've been burned by a silent string-vs-number bug in your own automations, reply — collecting these.

Facebook Post

Had a fun (read: mildly infuriating) bug this week building an n8n automation. Wired up a workflow so big deals ping the sales manager and small deals just get logged quietly — should've been a five-minute IF node setup.

Turns out the form was sending deal amounts as text (`"1000"`) instead of actual numbers, and the comparison silently returned the wrong answer. No error message, nothing red, just a workflow quietly doing the wrong thing.

Fixed it by forcing the field to a real number before comparing. New video + article walk through the whole thing, including why a "branch that didn't fire" isn't actually a bug — that one tripped me up the first time too. Link in comments if you want the full breakdown.

Reddit Summary

Wired up an n8n IF node to branch a workflow based on deal size — over a threshold pings Slack, under it just logs quietly. Ran into the classic string-vs-number trap: the incoming field was `"1000"` (string) instead of `1000` (number), and the numeric comparison silently evaluated wrong with zero error output.

Two things worth knowing if you're new to n8n's IF node:

1. A branch that doesn't fire shows no checkmark on its downstream nodes — that's expected, not a failure. Only the matching branch executes.
2. There's a "force type" option inside the condition specifically for the string/number mismatch case — worth turning on any time your trigger data comes from a web form, since those almost always send everything as strings.

Also worth knowing: IF is strictly two-output (true/false). Once you need 3+ outcomes off a single field, the Switch node is the right tool instead of chaining multiple IFs together.

Wrote up the full pattern with a Python equivalent (for anyone who wants to see the same logic outside of n8n) if useful — happy to answer questions about the setup in comments.

GitHub README

```
# Deal Routing with n8n's IF Node
```

```
## Problem
```

A single "new deal" trigger needs to produce different behavior depending on the deal's data: large deals should notify a sales manager immediately, small deals should be logged without interrupting anyone. A naive workflow that fires the same action for every deal regardless of size gets ignored.

```
## Intuition
```

Treat the condition like a bouncer at a door: one field, one comparison, one yes/no answer, then the workflow splits. Put the expensive/interrupting action strictly on the branch that fires only when it should be rare. The branch that doesn't fire shows no output — that's correct behavior, not a failure.

The most common silent bug: a field arriving as a string (`"1000"`) being compared against a number (`1000`). No error is thrown; the condition just evaluates incorrectly. Force the type before comparing.

```
## Approach
```

1. Define the condition: field, operator, comparison value.
2. Wire the true branch to the notification step (Slack, email).
3. Wire the false branch to a quiet logging step.
4. Stack conditions with AND/OR when one field isn't enough
(`deal\_amount > 1000 AND region == "US"`).
5. Explicitly force numeric type on fields that might arrive as strings.
6. Once a single field needs more than two outcomes, switch to the Switch node instead of chaining IF nodes.

```
## Python Solution

```python
from dataclasses import dataclass

@dataclass
class Deal:
 deal_amount: float
 region: str

def should_notify_manager(deal: Deal, threshold: float = 1000, target_region: str = "US") -> bool:
 amount = float(deal.deal_amount)
 return amount > threshold and deal.region == target_region

def route_deal(deal: Deal) -> str:
 return "notify" if should_notify_manager(deal) else "log"

def handle_deal(raw_payload: dict) -> str:
 deal = Deal(deal_amount=raw_payload["deal_amount"], region=raw_payload["region"])
 branch = route_deal(deal)
 if branch == "notify":
 send_slack_alert(deal)
 else:
 log_deal(deal)
 return branch

def send_slack_alert(deal: Deal) -> None:
 print(f"[SLACK] New deal ${deal.deal_amount} in {deal.region} needs review")

def log_deal(deal: Deal) -> None:
 print(f"[LOG] Deal recorded: ${deal.deal_amount} in {deal.region}")
```

## Complexity

---

- Time:  $O(1)$  per deal — fixed number of field checks and comparisons.
- Space:  $O(1)$  — no data structures scale with input size.

## Video

---

Full walkthrough, including the live true/false branch demo: [{LINK}](#)

## Article

---

Full written companion with dry run, edge cases, and common mistakes: see the linked article for this episode. ``

## Quick Review

---

### **n8n IF node: when to use it?**

When a workflow must branch into two paths based on a field condition — e.g., deal value above/below a threshold.

### **IF node condition anatomy: three parts?**

Left value (field/expression), operator (equals, contains, larger, etc.), right value (comparison target).

### **IF node outputs: what do they mean?**

Top output = condition true; bottom output = condition false. Wire different actions to each.

### **String vs Number trap in IF conditions?**

A numeric-looking string ('50') fails numeric comparisons unless the operand type matches — cast/convert before comparing.

### **More than two outcomes needed — IF vs Switch?**

IF only gives true/false; use the Switch node when a field can route to three or more distinct branches.

### **Interview follow-up: why guess the outcome before running?**

Forces you to trace the condition logic manually, catching type-mismatch or wrong-operator bugs before they run live.